SIMATS ENGINEERING

ASSESSMENT TOOL 2

Course Name	Artificial Intelligence	Course Code	CSA17
Name	Krise Monali	Register Number	192519111

Course Outcome Covered: CO2 – Search and game-solving techniques (Greedy Search, A*, CSP, Minimax, Alpha-Beta pruning) with heuristic functions (BL3)

Annexure A – Scenario Based Assignment

Code of Conduct: I, Krise Monali (Reg No: 192519111), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Question 1 – AI Drones for Emergency Medicine Delivery (Greedy vs A*)

Problem model. States are locations on a map (graph); actions are flights between waypoints; step cost g(n) varies with terrain and weather; the heuristic h(n) is straight-line (Euclidean) distance to the destination. Edges may become blocked at run time, and only partial updates are available.

i. Greedy Best-First Search (GBFS)

GBFS expands the node with the lowest h(n) only, so f(n) = h(n). The drone always moves toward whichever waypoint looks geometrically closest to the destination, ignoring the cost already spent and the weather/terrain cost ahead.

Strengths:

- Very fast decisions and low computation: it usually expands few nodes, which suits urgent, real-time use.
- Low memory use compared with exhaustive search.
- Reacts quickly when the heuristic is informative, e.g. open terrain with few obstacles.

Weaknesses under uncertainty:

- Not optimal: a route that looks straight may cross storm zones or costly terrain.
- Not complete in graphs with loops unless a visited list is kept; it can get trapped in dead ends (a "U-shaped" flooded area where the straight line leads to a blocked river).
- Blind to path cost, so it ignores risk (high winds, rough terrain).
- Misleading heuristic: straight-line distance underestimates real distance when roads are blocked, so GBFS commits to bad paths and must backtrack.

ii. A* Search

A* uses f(n) = g(n) + h(n), where g(n) is the actual cost so far (including terrain and weather-dependent costs) and h(n) is the estimated cost remaining. The g(n) term keeps A* from following paths that are cheap-looking but already expensive; the h(n) term focuses the search toward the goal instead of expanding in all directions like Uniform Cost Search.

- If h(n) is **admissible** (never overestimates the true cost) and **consistent**, A* is complete and returns the optimal (least-cost) path. Straight-line distance is admissible when the cost per unit distance is at least 1 (or if costs are scaled by the minimum cost per unit distance).

- Balance: when weather makes an edge costly, g grows quickly, so A* shifts to alternatives; when g is similar everywhere, h drives it toward the goal. A weighted variant, $f = g + w \cdot h$ ($w > 1$), trades some optimality for speed when time is critical.
- Cost: A* expands more nodes and needs more memory than GBFS, which can be a concern on large maps.

iii. Impact of heuristic accuracy

Heuristic quality	Greedy Best-First	A*
Accurate (close to true cost)	Very fast, near-direct path, often good	Expands very few nodes, optimal
Underestimate (admissible but weak)	Wanders; may pick poor routes	Still optimal but explores more nodes (approaches Uniform Cost Search as $h \to 0$)
Overestimate (inadmissible)	Little change in behaviour, but misleads ordering	May lose optimality and skip the best path
Misleading (ignores blocked roads/weather)	Can get stuck in dead ends, large detours	Cost term g(n) partly corrects errors, so it degrades more gracefully

The key point: GBFS depends *entirely* on h, so errors in h directly cause bad routes. A* is protected by g(n) and by the admissibility guarantee. A better heuristic (e.g. distance adjusted for known weather/terrain factors) improves both.

iv. Effect of dynamic changes (blocked paths)

- When a path becomes blocked, the stored plan becomes invalid. The drone must **replan**.
- GBFS replans cheaply but may repeatedly walk into new dead ends because it has no cost memory; quality of the final route can be poor.
- A* replanning from scratch is costlier, but it yields an optimal route on the updated map. Efficiency can be improved using **incremental/anytime variants**: D* Lite, Lifelong Planning A*, or Anytime Repairing A* (ARA*), which reuse earlier search effort instead of starting over.
- Partial observability means updates may be incomplete, so the drone should plan on the best known map and re-plan when new information arrives (receding-horizon planning). Frequent changes increase computational load and may cause oscillation between routes; a minimum-improvement threshold avoids flip-flopping.

v. Recommendation

Recommended: A* (preferably the incremental/replanning form such as D* Lite, possibly with weighted A* when time is very tight).

- *Optimality and risk:* A* accounts for variable cost (terrain, weather), giving the least-delay, least-risk route; GBFS cannot.
- *Safety:* the cost model can include risk penalties (high-wind zones, no-fly areas) so dangerous routes are avoided; GBFS ignores them.
- *Real-time constraints:* use weighted A* or anytime A* to return a good path quickly and improve it while flying, and use D* Lite to repair the path when roads close.
- *Fallback:* GBFS can be used for a very quick initial path when computation time is extremely limited, but should not be the final decision strategy for medicine delivery where failure is costly.

Question 2 – Smart City Traffic Signal Optimization

Formulation as an optimization problem.

- *State:* the full set of signal timing parameters, i.e. cycle length, green-phase splits and offsets for every intersection (hundreds of variables).

- *Objective function:* minimize F = w1·(total waiting time) + w2·(fuel consumption) + w3·(congestion index), with weights w1, w2, w3 set by city policy.
- *Neighbourhood/actions:* change one intersection's green duration or offset by a small amount.
- *Constraints:* minimum green time, pedestrian crossing time, maximum cycle length, safe phase ordering.
- *Evaluation:* a traffic simulator or real-time sensor data gives F for a configuration.

Why traditional search is inefficient. With, say, 500 intersections and 20 possible timings per intersection, there are 20^500 configurations. BFS/DFS/A* would need to store and explore an astronomically large tree, and there is no clear goal state or path cost: only a quality score. Traffic also changes continuously, so any precomputed solution quickly goes stale. This is why *local search* (which keeps only the current state and improves it) is suited.

i. Hill Climbing

Start with the current signal plan (or a random one), evaluate neighbours (adjust the timing at one or a few intersections), move to the neighbour that gives the best improvement in F, and repeat until no neighbour is better.

Advantages: simple, very low memory (stores only the current state), fast improvement, suitable for continuous online adjustment.

Limitations: gets stuck at local optima, plateaus and ridges; results depend on the starting point; no backtracking; cannot escape poor regions; it is greedy and incomplete.

ii. Local maxima, plateaus and ridges (real-world meaning)

- **Local maximum (here, local minimum of cost):** a configuration in which any single-intersection change makes traffic worse, yet a better global arrangement exists. Example: each junction is tuned well for itself, but a "green wave" along a corridor needs several signals changed together.
- **Plateau:** many neighbouring settings give the same total waiting time, e.g. in light traffic small changes in green time make no measurable difference, so the algorithm has no direction to move and can wander.
- **Ridge:** the improvement direction needs *simultaneous* changes in multiple variables, which single-step moves cannot follow. Example: improving flow on an arterial road needs offset changes on a chain of consecutive signals at once; changing any one alone makes it worse.

iii. Simulated Annealing (SA) and Genetic Algorithms (GA)

Simulated Annealing: picks a random neighbour; always accepts it if better, and accepts a worse move with probability e^(−ΔE/T). The temperature T starts high (lots of exploration) and is lowered gradually (cooling schedule), so the search escapes local optima early and then settles into a good solution. With a slow enough schedule it can converge to the global optimum.

Genetic Algorithm: maintains a *population* of signal plans (chromosomes = timing parameters). Fitness = inverse of F. Selection picks the better plans, crossover combines parts of two plans (e.g. corridors from different parents), and mutation randomly changes timings. Because it evaluates many candidates in parallel, it explores widely and can combine good "building blocks" (good corridor settings), handling ridges well.

Both overcome hill climbing's limitation by introducing randomness and accepting temporary worsening (SA) or population diversity (GA).

iv. Recommendation

Recommended: a hybrid approach: Genetic Algorithm (or population-based search) for global optimization/offline planning of base timing plans, followed by Simulated Annealing or hill climbing for online fine-tuning as traffic changes.

- *Scalability:* GA evaluations are naturally parallelizable across the population (and across city zones); the problem can be decomposed by district. SA has tiny memory use and scales easily.

- *Adaptability:* when traffic changes, re-seed the GA population with the current best plans and run a few generations; or let SA do small adjustments with a low temperature. This gives continuous adaptation without restarting.
- *Quality:* avoids local optima, which pure hill climbing cannot.
- Use random-restart hill climbing only as a cheap baseline.

Question 3 – Autonomous Mars Rover (Online Search)

i. Why online search is required

Offline search computes the entire plan before acting and assumes a known, deterministic map. The rover has no complete map, and communication delay with Earth (minutes of latency) prevents remote step-by-step control. Online search **interleaves planning and acting**: the agent chooses an action, executes it, observes the percept, and then decides the next action. It is needed because (a) states and action outcomes are unknown until explored, (b) the environment reveals hazards only as sensors detect them, and (c) the cost of exploring unsafely is irreversible (damage).

ii. Partial observability and dynamic environment: challenges

- **Limited sensing:** the rover sees only nearby terrain, so it may miss craters or rocks beyond range; sensor noise (dust, shadows, glare) causes misclassification.
- **Unknown map:** the heuristic (distance to the sample site) may be misleading, e.g. a straight path blocked by a ravine.
- **Irreversible actions / dead ends:** falling into a crater or getting stuck cannot be undone (unsafe exploration), so the agent must prefer reversible actions.
- **Localization error:** wheel slip and drift make position estimates inaccurate (needs SLAM / sensor fusion).
- **Dynamic factors:** dust storms, changing light, shifting sand and falling battery/energy levels change costs and safe regions over time.
- **Computation and energy limits:** onboard processing and power are limited, so heavy planning must be bounded.

iii. Building and updating the internal model

1. **Initial model:** whatever prior data exists (orbital images, elevation maps) forms a coarse occupancy/cost map with unknown cells.
2. **Perception:** cameras, LiDAR, inertial sensors and wheel odometers provide percepts.
3. **Mapping and localization (SLAM):** the rover estimates its pose and adds new cells to the map (free, obstacle, hazard, unknown), often with probabilities (a Bayesian/occupancy-grid update, belief state).
4. **Model update:** for every visited state it stores the observed result of each action (as in online agents such as LRTA*: it keeps a table H(s) of the learned cost-to-goal and updates it as it moves, plus a record of which actions were tried at each state).
5. **Replanning:** after each update, plan again over the updated model (e.g. D* Lite or LRTA*) and execute the next safe step. Newly discovered hazards raise the cost of cells or mark them as blocked.

iv. Comparison: online vs uninformed vs informed search

Aspect	Uninformed (BFS/DFS/UCS)	Informed (Greedy/A*)	Online (LRTA*, D* Lite)
Knowledge needed	Complete map	Complete map + heuristic	Partial map; learns while moving
When it plans	Offline, before acting	Offline (can be re-run)	Interleaved with acting

Aspect	Uninformed (BFS/DFS/UCS)	Informed (Greedy/A*)	Online (LRTA*, D* Lite)
Efficiency	Low; explores blindly	High given a good heuristic	Efficient: explores only what is needed
Unknown/dynamic terrain	Cannot handle	Only with frequent re-running	Yes; designed for it
Optimality	BFS/UCS optimal on known graph	A* optimal on known graph	Not guaranteed; measured by competitive ratio
Safety in unknown terrain	Not applicable	Plans assuming the map is correct	Can use cautious, reversible exploration

Uninformed and informed algorithms assume that the state space is fully known; online search accepts that it is not and trades guaranteed optimality for the ability to act.

v. Most suitable approach

Recommended: an informed *online* search with replanning (D* Lite or LRTA*-style learning) combined with a safety-aware cost map and SLAM.

- *Uncertainty:* a belief/occupancy map and probabilistic updates handle noisy, partial sensing.
- *Adaptability:* D* Lite repairs the path incrementally as new obstacles are discovered, much cheaper than replanning from scratch; the heuristic (distance to the sample site) guides it efficiently.
- *Safety:* very high costs (or forbidden status) for crater edges, steep slopes and unknown cells; prefer reversible moves; keep an escape route/return path; stop and wait if confidence is low.
- *Resource limits:* bounded-time planning with limited lookahead suits onboard computation.

Question 4 – University Exam Timetable (CSP)

CSP formulation

A CSP is defined by **variables, domains, and constraints**.

i. Variables, domains and constraints

Variables: one variable per exam, E1, E2, ..., En (one per course), whose value is its (time slot, hall) assignment. (Alternatively two variables per exam: Time(Ei) and Hall(Ei).)

Domains: Time(Ei) ∈ {all available time slots in the exam period, e.g. 3 slots a day × 10 days = 30 slots}; Hall(Ei) ∈ {Hall 1, ..., Hall m}. Domains are reduced up front for faculty unavailability and special requirements (e.g. an exam needing a computer lab has only lab halls in its domain).

Constraints:

Constraint	Type	Explanation
No student has overlapping exams	Binary	If courses Ei and Ej share any student, then Time(Ei) ≠ Time(Ej)
Limited exam halls	Global/capacity	At most m exams in the same slot; exams in the same slot need different halls, and hall capacity ≥ number of students
Precedence (some subjects before others)	Binary ordering	Time(Ei) < Time(Ej) for prerequisite pairs
Faculty availability	Unary	Time(Ei) must be in the slots where the invigilator/examiner is available
Special accommodations	Unary/binary	e.g. extra time (needs a longer slot), separate hall, ground-floor hall, or no two consecutive exams for the student

A *hard* constraint must always hold; *soft* preferences (spreading exams for each student, avoiding early-morning slots) can be added as costs, turning it into a constraint optimization problem. A **solution** is a complete assignment of values to all variables that satisfies every constraint.

ii. Backtracking search

Backtracking is depth-first search that assigns one variable at a time and checks consistency after each assignment:

1. Choose an unassigned variable (an exam).
2. Try each value in its domain (slot, hall).
3. If the assignment violates any constraint with those already assigned, discard it; otherwise recurse.
4. If no value works, **backtrack** to the previous variable and try the next value.
5. Stop when all exams are assigned (success) or all options are exhausted (failure).

It prunes a whole subtree as soon as a partial assignment is inconsistent, so it is far better than brute-force generate-and-test. However, naive backtracking can still do exponential work, thrashing on bad early choices.

iii. Constraint propagation and forward checking

Forward checking: after assigning a value to an exam, remove inconsistent values from the domains of the unassigned exams related by constraints. E.g. after fixing Exam A (Maths) in Slot 3, remove Slot 3 from every exam that shares students with it. If any domain becomes empty, backtrack immediately, so failures are detected *early*, before going deeper.

Stronger propagation: *arc consistency (AC-3)* repeatedly removes values that have no supporting value in a neighbouring variable's domain, and so detects further inconsistencies, e.g. in precedence chains.

Heuristics that work with it:

- *MRV (Minimum Remaining Values):* choose the exam with the fewest legal slots (most constrained first).
- *Degree heuristic:* break ties by choosing the exam involved in the most conflicts (largest number of shared students).
- *LCV (Least Constraining Value):* try the slot that rules out the fewest options for others.

Together, these drastically reduce the number of nodes explored.

iv. Real-world challenges and best strategy for scalability

Challenges: thousands of exams and students make the conflict graph dense; the problem is NP-hard (related to graph colouring); constraints may be over-constrained (no perfect solution exists); last-minute changes (faculty unavailability, new accommodations) need quick repair; soft preferences conflict with hard rules; fairness and student well-being.

Recommended strategy: a **hybrid**:

1. Model as a CSP and reduce the domains with preprocessing (unary constraints, arc consistency).
2. Use backtracking with MRV + degree + LCV and forward checking/AC-3 (maintaining arc consistency) to get a feasible timetable.
3. For large instances, build the initial solution with a graph-colouring heuristic (e.g. DSATUR, which is MRV-like) and improve it with **local search (min-conflicts)**, tabu search or simulated annealing, which also handle soft preferences and re-scheduling after changes.
4. Decompose the problem (by department/exam day) and use intelligent backtracking (conflict-directed backjumping) to avoid thrashing.

This offers completeness for hard constraints where possible, scalability from local search, and quick repair when conditions change.

Question 5 – Real-Time Strategy Game AI (Minimax and Alpha-Beta)

i. How Minimax models the problem

Minimax models the game as a **two-player, zero-sum, adversarial search** problem with a game tree:

- *States:* game positions (units, resources, map control).
- *Players:* MAX (the AI) and MIN (the human opponent).
- *Actions:* legal moves for the player to move.
- *Terminal test and utility:* win/loss/draw values (+1, −1, 0 or a score).
- *Algorithm:* explore the tree to a depth limit; at leaf nodes use utility or the evaluation function; at MAX nodes choose the child with the **maximum** value, at MIN nodes the **minimum** (assuming the opponent plays optimally). Values propagate up to the root, and the AI picks the move with the best guaranteed outcome against best play. (An RTS is not strictly turn-based, so it is approximated by discretising time into decision ticks or planning on abstracted states.)

ii. Computational challenges of large game trees

- The tree has b^d nodes (branching factor b, depth d). In an RTS with dozens of units, b can reach hundreds or thousands, so even d = 4 to 6 gives billions of nodes.
- Time limits are strict (milliseconds per decision), so full-depth search is impossible; a depth cutoff means reliance on heuristic evaluation, which can be inaccurate (horizon effect).
- Real-time play, simultaneous moves, imperfect information (fog of war) and randomness break the pure minimax assumptions.
- Memory use and repeated states (transpositions) multiply the work.

iii. How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning returns **exactly the same result as Minimax** but skips branches that cannot affect the final decision.

- **Alpha (α)** = best (highest) value MAX can guarantee so far on the path to the root.
- **Beta (β)** = best (lowest) value MIN can guarantee so far.
- At any node, if **α ≥ β**, the remaining children are pruned: the opponent would never allow the game to reach this branch.

Example reasoning: MAX has found a move guaranteeing 6 (α = 6). Exploring a second move, MIN's first reply gives 4. MIN can force at most 4 there, which is worse than 6 for MAX, so the remaining replies need not be examined; MAX will not choose this move.

Efficiency: in the worst case (bad move ordering), no pruning occurs, O(b^d). With **best move ordering** the complexity becomes about O(b^(d/2)), i.e. the search can go roughly **twice as deep** in the same time. With random ordering, roughly O(b^(3d/4)).

Improving pruning: order moves by promising value (killer moves, history heuristic, previous-iteration best move first), use transposition tables, and use iterative deepening.

iv. Evaluation function design

An evaluation function estimates how good a non-terminal state is for MAX. A common form is a weighted linear sum:

Eval(s) = w1·f1 + w2·f2 + ... + wn·fn

Feature	Reason
Material/army strength (units' health, attack, cost)	Direct measure of fighting power
Resource income and stockpile (gold, wood, etc.)	Economy supports future production

Feature	Reason
Map and objective control (bases, key locations)	Positional advantage and vision
Technology/upgrade level	Long-term advantage
Unit safety/positioning (distance to enemy, formation, threat level)	Avoids immediate losses
Production capacity	Ability to replace losses
Opponent's weakness (e.g. exposed base), with negative sign	Adversarial balance

Weights are set by expert knowledge or learned (machine learning, self-play, TD-learning). The function must be **fast to compute** and correlated with the real chance of winning, and values for terminal states must be consistent with utility (win = highest).

v. Strategy for balancing optimality and real-time performance

- **Iterative deepening with a time budget:** search depth 1, 2, 3... and return the best move from the deepest completed iteration when time runs out (anytime behaviour).
- **Alpha-Beta with good move ordering and transposition tables** to maximise depth.
- **Depth-limited search with a good evaluation function** and quiescence search to avoid the horizon effect in volatile positions.
- **Selective/forward pruning:** consider only the top-k moves, or use abstractions (group units, high-level commands) to lower b.
- **Monte Carlo Tree Search (MCTS)** or a hybrid with alpha-beta when branching is huge or information is hidden.
- **Precomputation:** opening books, cached patterns, and parallel search on multiple cores.
- Accept near-optimal (not provably optimal) moves: a slightly weaker move delivered on time is better than a perfect move that arrives late.

Submitted by: Krise Monali | **Register Number:** 192519111